

HACETTEPE UNIVERSITY

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

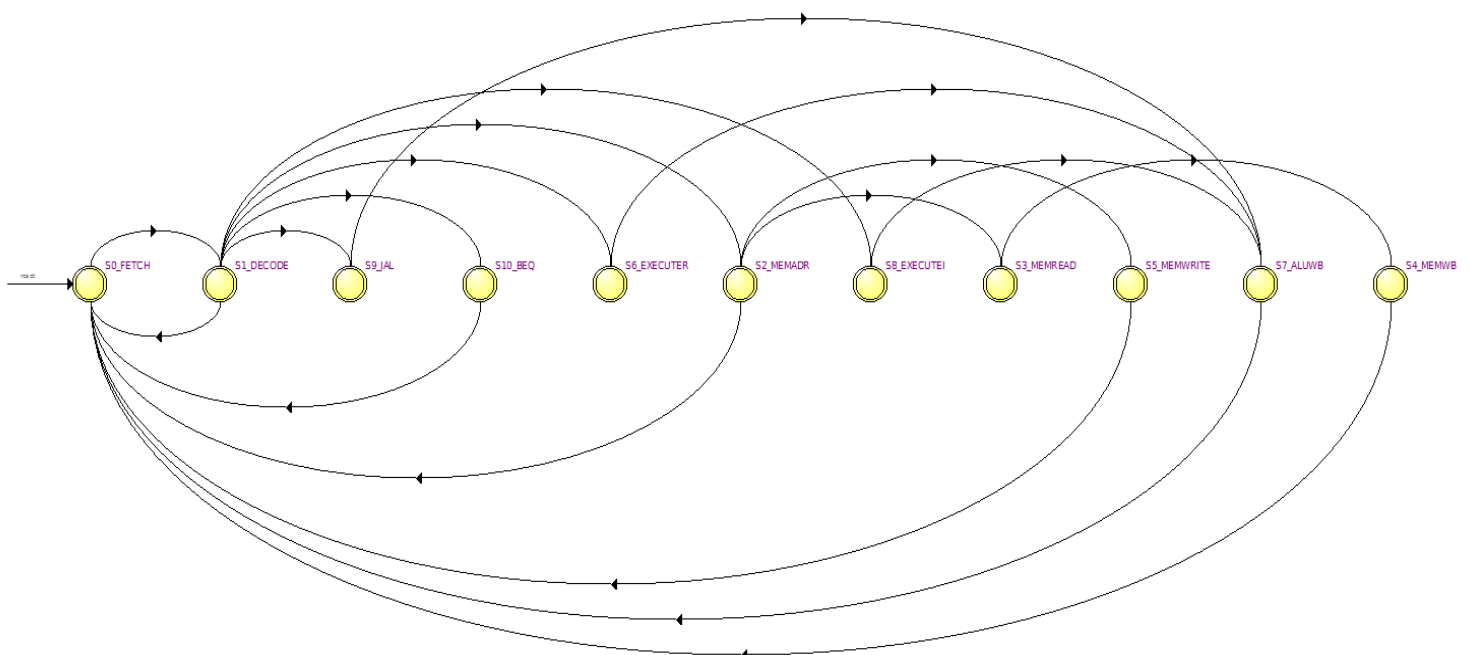
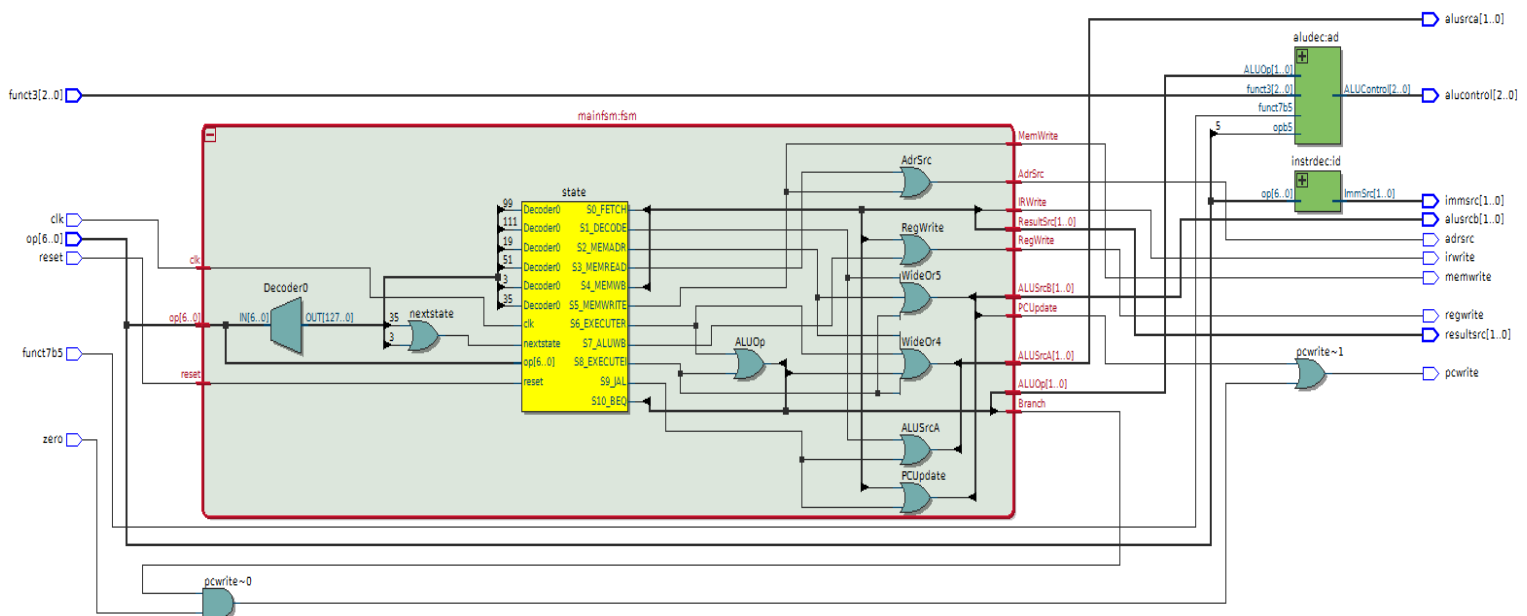
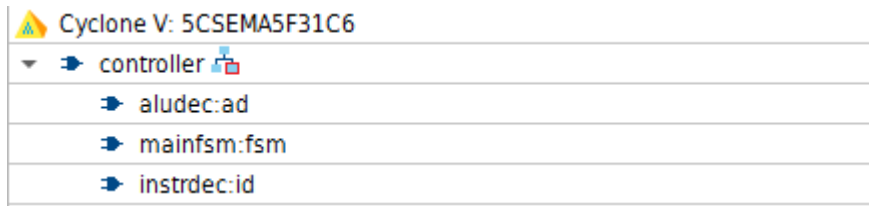
ELE 432: Advanced Digital Design

Homework 2

Hakan Töre
2210357024

- 1) I have spent 5 hours on this homework. Although most of the time was spent on figuring out why the ALUControl values were off in the questa transcript. The controller.tv test vector file was wrong and did not match the architecture given to us by this course so we had to update the test vectors.

2)



```

/*
Hakan Töre 2210357024
ELE432 Homework 2 : HW2_Multi-Cycle_RISC-V_Processor
controller.sv
*/
module controller(
    input logic          clk,
    input logic          reset,
    input logic  [6:0] op,
    input logic  [2:0] funct3,
    input logic          funct7b5,
    input logic          zero,
    output logic [1:0] immsrc,
    output logic [1:0] alusrca, alusrcb,
    output logic [1:0] resultsrc,
    output logic          adrsrc,
    output logic [2:0] alucontrol,
    output logic          irwrite, pcwrite,
    output logic          regwrite, memwrite
);

    logic [1:0] aluop;
    logic      branch;
    logic      pcupdate;

    assign pcwrite = pcupdate | (branch & zero);

    instrdec id (
        .op(op),
        .ImmSrc(immsrc)
    );

    aludec ad (
        .opb5(op[5]),
        .funct3(funct3),
        .funct7b5(funct7b5),
        .ALUOp(aluop),
        .ALUControl(alucontrol)
    );

    mainfsm fsm (
        .clk(clk),
        .reset(reset),
        .op(op),
        .Branch(branch),
        .PCUpdate(pcupdate),
        .RegWrite(regwrite),
        .MemWrite(memwrite),
        .IRWrite(irwrite),
        .ResultSrc(resultsrc),
        .ALUSrcA(alusrca),
        .ALUSrcB(alusrcb),
        .AdrSrc(adrsrc),
        .ALUOp(aluop)
    );

endmodule

```

```

/*
    Hakan Töre 2210357024
    ELE432 Homework 2 : HW2_Multi-Cycle_RISC-V_Processor
    mainfsm.sv
*/

module mainfsm (
    input logic      clk,
    input logic      reset,
    input logic [6:0] op,
    output logic      Branch,
    output logic      PCUpdate,
    output logic      RegWrite,
    output logic      MemWrite,
    output logic      IRWrite,
    output logic [1:0] ResultSrc,
    output logic [1:0] ALUSrcA,
    output logic [1:0] ALUSrcB,
    output logic      AdrSrc,
    output logic [1:0] ALUOp
);

    // State Encoding
    typedef enum logic [3:0] {
        S0_FETCH    = 4'b0000,
        S1_DECODE    = 4'b0001,
        S2_MEMADR    = 4'b0010,
        S3_MEMREAD   = 4'b0011,
        S4_MEMWB     = 4'b0100,
        S5_MEMWRITE  = 4'b0101,
        S6_EXECUTER  = 4'b0110,
        S7_ALUWB     = 4'b0111,
        S8_EXECUTEI  = 4'b1000,
        S9_JAL       = 4'b1001,
        S10_BEQ      = 4'b1010
    } statetype;

    statetype state, nextstate;
    // State Register
    always_ff @(posedge clk or posedge reset) begin
        if (reset) state <= S0_FETCH;
        else      state <= nextstate;
    end

    // Next State Logic
    always_comb begin
        case (state)
            S0_FETCH:    nextstate = S1_DECODE;
            S1_DECODE: begin
                case (op)
                    7'b0000011: nextstate = S2_MEMADR; // lw
                    7'b0100011: nextstate = S2_MEMADR; // sw
                    7'b0110011: nextstate = S6_EXECUTER; // R-type
                    7'b0010011: nextstate = S8_EXECUTEI; // I-type ALU
                    7'b1101111: nextstate = S9_JAL; // jal
                    7'b1100011: nextstate = S10_BEQ; // beq
                    default:    nextstate = S0_FETCH;
                endcase
            end
            S2_MEMADR: begin
                case (op)
                    7'b0000011: nextstate = S3_MEMREAD; // lw
                    7'b0100011: nextstate = S5_MEMWRITE; // sw
                    default:    nextstate = S0_FETCH;
                endcase
            end
            S3_MEMREAD: nextstate = S4_MEMWB;
            S4_MEMWB:    nextstate = S0_FETCH;
            S5_MEMWRITE: nextstate = S0_FETCH;
            S6_EXECUTER: nextstate = S7_ALUWB;
            S7_ALUWB:    nextstate = S0_FETCH;
            S8_EXECUTEI: nextstate = S7_ALUWB;
            S9_JAL:      nextstate = S7_ALUWB;
            S10_BEQ:     nextstate = S0_FETCH;
            default:     nextstate = S0_FETCH;
        endcase
    end
end

```

```

// Output Logic
always_comb begin
    // When outputs are don't care, set them to 0
    Branch    = 1'b0;
    PCUpdate  = 1'b0;
    RegWrite  = 1'b0;
    MemWrite  = 1'b0;
    IRWrite   = 1'b0;
    ResultSrc = 2'b00;
    ALUSrcA   = 2'b00;
    ALUSrcB   = 2'b00;
    AdrSrc    = 1'b0;
    ALUOp     = 2'b00;
    case (state)
        S0_FETCH: begin
            AdrSrc    = 1'b0;
            IRWrite   = 1'b1;
            ALUSrcA   = 2'b00;
            ALUSrcB   = 2'b10;
            ALUOp     = 2'b00;
            ResultSrc = 2'b10;
            PCUpdate  = 1'b1;
        end
        S1_DECODE: begin
            ALUSrcA   = 2'b01;
            ALUSrcB   = 2'b01;
            ALUOp     = 2'b00;
        end
        S2_MEMADR: begin
            ALUSrcA   = 2'b10;
            ALUSrcB   = 2'b01;
            ALUOp     = 2'b00;
        end
        S3_MEMREAD: begin
            ResultSrc = 2'b00;
            AdrSrc    = 1'b1;
        end
        S4_MEMWB: begin
            ResultSrc = 2'b01;
            RegWrite  = 1'b1;
        end
        S5_MEMWRITE: begin
            ResultSrc = 2'b00;
            AdrSrc    = 1'b1;
            MemWrite  = 1'b1;
        end
        S6_EXECUTER: begin
            ALUSrcA   = 2'b10;
            ALUSrcB   = 2'b00;
            ALUOp     = 2'b10;
        end
        S7_ALUWB: begin
            ResultSrc = 2'b00;
            RegWrite  = 1'b1;
        end
        S8_EXECUTEI: begin
            ALUSrcA   = 2'b10;
            ALUSrcB   = 2'b01;
            ALUOp     = 2'b10;
        end
        S9_JAL: begin
            ALUSrcA   = 2'b01;
            ALUSrcB   = 2'b10;
            ALUOp     = 2'b00;
            ResultSrc = 2'b00;
            PCUpdate  = 1'b1;
        end
        S10_BEQ: begin
            ALUSrcA   = 2'b10;
            ALUSrcB   = 2'b00;
            ALUOp     = 2'b01;
            ResultSrc = 2'b00;
            Branch    = 1'b1;
        end
    endcase
end
endmodule

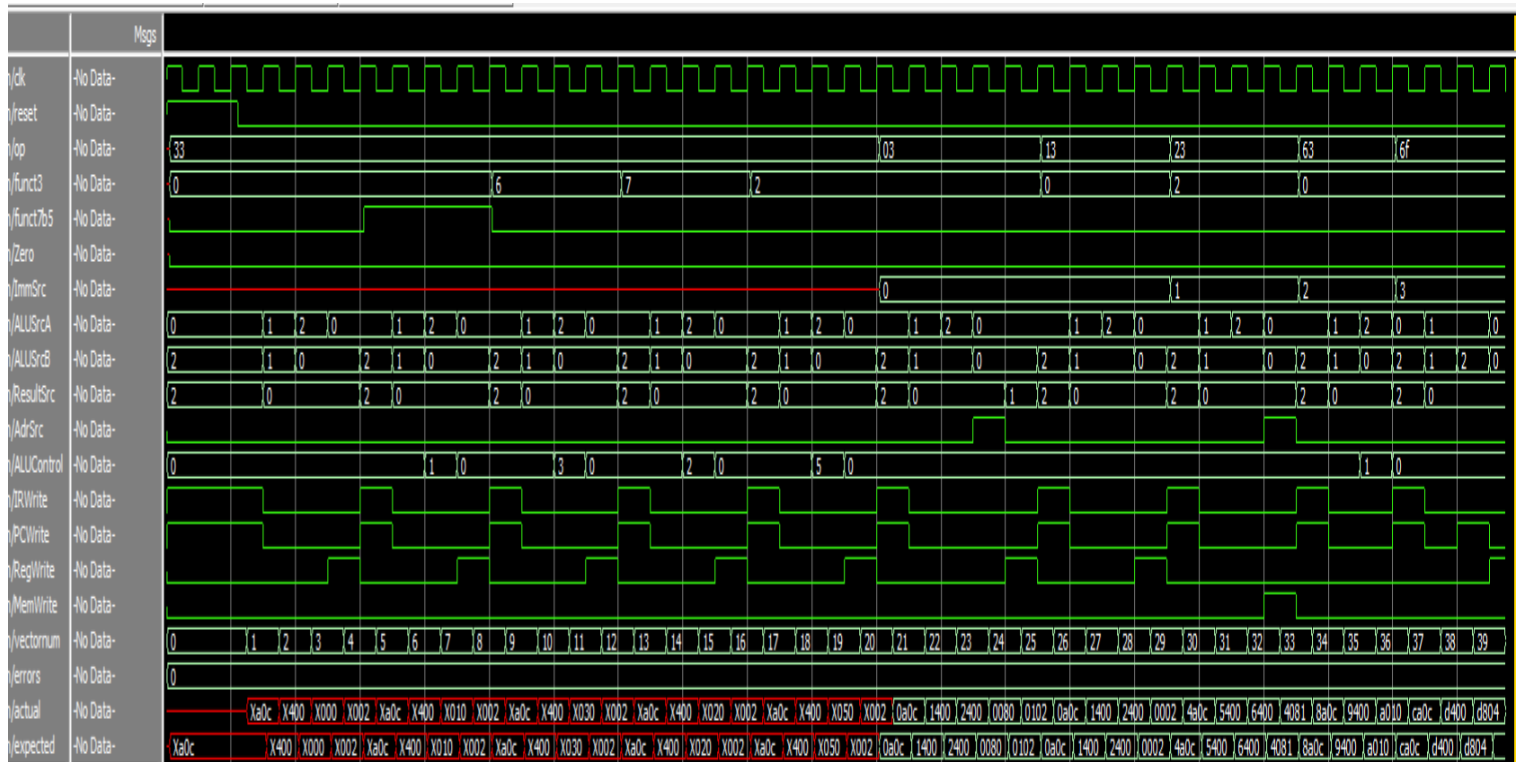
```

aludec.sv instrdec.sv and controller testbench.sv were given to us by default there were no changes done to them but I have changed the test vectors controller.tv

- 3) The controller.tv file was using a RISC-V standard different than the one in the Harris & Harris book and the homework file. I updated the expected ALUControl bits to match the Harris & Harris's description. The updated file can be found in the GitHub repository.

Example:

<pre>//op[6:0]_funct3[2:0]_funct7b5_Zero_ImmSrc[1:0]_ALUSrcA[1:0]_ALUSrcB[1:0]_ResultSrc[1:0]_AdrSrc_ALUControl[2:0]_IRWr ite_PCWrite_RegWrite_MemWrite // add</pre>	
old	updated
0110011_000_0_0_XX_00_10_10_0_010_1_1_0_0 // Fetch	0110011_000_0_0_XX_00_10_10_0_000_1_1_0_0 // Fetch
0110011_000_0_0_XX_01_01_00_0_010_0_0_0_0 // Decode	0110011_000_0_0_XX_01_01_00_0_000_0_0_0_0 // Decode
0110011_000_0_0_XX_10_00_00_0_010_0_0_0_0 // ExecuteR	0110011_000_0_0_XX_10_00_00_0_000_0_0_0_0 // ExecuteR
0110011_000_0_0_XX_00_00_00_0_010_0_0_1_0 // ALUWB	0110011_000_0_0_XX_00_00_00_0_000_0_0_1_0 // ALUWB



As you can see from the error counter there is no errors

```
# run -all
#          40 tests completed with          0 errors
# ** Note: $stop      : C:/Users/Casper/AppData/Local/quartus_4_mc_processor/controller_testbench.sv(81)
#   Time: 415 ps  Iteration: 1  Instance: /controller_testbench
# Break in Module controller testbench at C:/Users/Casper/AppData/Local/quartus_4_mc_processor/controller_testbench.sv line 81
```

- 4)

<https://github.com/Hakan-26/-Multicycle-Processor-Controller>